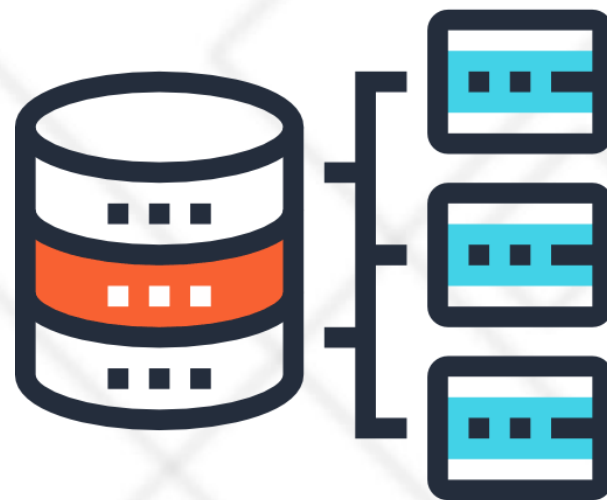




北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY



# 第4章 关系数据库 标准语言SQL

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn



# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
  - 4.4.1 数据查询
  - 4.4.2 数据更新
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结

# 1. 插入数据

## □ 两种插入数据方式

- 插入单个元组
- 插入子查询结果

### (1) 插入单个元组

- 语句格式

**INSERT INTO <表名> [(<属性列1>[, <属性列2> ...])]**

**VALUES (<常量1> [, <常量2>] ...);**

- 功能：将新元组插入指定表中

# (1) 插入单个元组

**INSERT INTO <表名> [(<属性列1>[, <属性列2> ...])]  
VALUES (<常量1> [, <常量2>] ...);**

- INTO子句
  - ◆ 指定要插入数据的表名及属性列
  - ◆ 属性列的顺序可以与表定义中的顺序不一致
  - ◆ 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
  - ◆ 指定部分属性列：插入的元组在其余属性列上取空值
- VALUES子句
  - ◆ 提供的值必须与INTO子句匹配：值的类型和个数

# (1) 插入单个元组

- [例] 在学生表中插入一个学生元组，其学号为101215，姓名为李斌，男，19岁，是CS系的学生

**INSERT INTO Student**

**VALUES ('101215', '李斌', '男', 19, 'CS');**

- [例] 学号为101253的学生选了2号课，将其插入选课表中

**INSERT INTO SC(Sno, Cno)**

**VALUES ('101253', '2');**

- 新插入的记录在Grade列上取空值

**INSERT INTO SC**

**VALUES ('101253', '2', NULL);**

## (2) 插入子查询结果

□将子查询结果插入指定表中

**INSERT INTO <表名> [(<属性列1>[, <属性列2 > ...])]  
<SELECT语句>;**

□INTO子句

- 指定要插入数据的表名及属性列
- 属性列的顺序可与表定义中的顺序不一致
  - ◆ 没有指定属性列：表示要插入的是一条完整的元组
  - ◆ 指定部分属性列：插入的元组在其余属性列上取空值

□子查询

- SELECT子句目标列必须与INTO子句匹配：值的类型和个数

## (2) 插入子查询结果

□[例] 对每一个系，求学生的平均年龄，并把结果存入数据库

- 第一步：建表

```
CREATE TABLE Dept_age
```

```
(Sdept CHAR(15)      /* 系名*/
```

```
Avg_age SMALLINT); /*学生平均年龄*/
```

- 第二步：插入数据

```
INSERT INTO Dept_age(Sdept, Avg_age)
```

```
SELECT Sdept, AVG(Sage)
```

```
FROM Student
```

```
GROUP BY Sdept;
```

## (2) 插入子查询结果

- DBMS在执行插入语句时会检查所插元组是否破坏表上已定义的完整性规则
  - 实体完整性
  - 参照完整性
  - 用户定义的完整性
    - ◆ 对于有NOT NULL约束的属性列是否提供了非空值
    - ◆ 对于有UNIQUE约束的属性列是否提供了非重复值
    - ◆ 对于有值域约束的属性列所提供的属性值是否在值域范围内



## 2. 修改数据

### □ 语句格式

**UPDATE <表名>**

**SET 列名1=<表达式1>[, 列名2=<表达式2>] ...**

**[WHERE <条件表达式>];**

□ 功能：修改指定表中满足WHERE子句条件的元组

□ SET子句：指定修改方式

- 要修改的列
- 修改后取值

□ WHERE子句：指定要修改的元组

- 缺省表示要修改表中的所有元组

## 2. 修改数据

### □ 三种修改方式

#### ① 修改某一个元组的值

**[例]** 将学生101215的年龄改为22岁

```
UPDATE Student  
SET Sage=22  
WHERE Sno='101215';
```

#### ② 修改多个元组的值

**[例]** 将所有学生的年龄增加1岁

```
UPDATE Student SET Sage=Sage+1;
```

**[例]** 将信息系所有学生的年龄增加1岁

```
UPDATE Student SET Sage=Sage+1  
WHERE Sdept='IS';
```

## 2. 修改数据

### □ 三种修改方式 (续)

#### ③ 带子查询的修改语句

**[例]** 将计算机科学系全体学生的成绩置零

```
UPDATE SC
  SET Grade=0
  WHERE 'CS'=
    (SELETE Sdept
     FROM Student
     WHERE Student.Sno=SC.Sno);
```

## 2. 修改数据

- DBMS在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则
  - 实体完整性
  - 主键不允许修改
  - 用户定义的完整性
    - ◆ NOT NULL约束
    - ◆ UNIQUE约束
    - ◆ 值域约束

# 3. 删除数据

## □ 语句格式

**DELETE FROM <表名>**  
**[WHERE <条件>];**

## □ 功能

- 删除指定表中满足WHERE子句条件的元组

## □ WHERE子句

- 指定要删除的元组
- 缺省表示要删除表中的所有元组

# 3. 删除数据

## □ 三种删除方式

### ① 删除某一个元组的值

**[例]** 删除学号为201225的学生记录

**DELETE FROM Student**

**WHERE Sno='201225';**

### ② 删除多个元组的值

**[例]** 删除2号课程的所有选课记录

**DELETE FROM SC**

**WHERE Cno='2';**

**[例]** 删除所有的学生选课记录

**DELETE FROM SC;**

# 3. 删除数据

## □ 三种删除方式 (续)

### ③ 带子查询的删除语句

**[例]** 删除计算机科学系所有学生的选课记录

```
DELETE
FROM SC
WHERE 'CS'=
  (SELETE Sdept
   FROM Student
   WHERE Student.Sno=SC.Sno);
```

## □ DBMS在执行增删改语句时必须保证数据库一致性

### ● 数据库不一致性的形成

◆ 当某同学退学时，首先删除学生表中数据，然后删除选课表中的数据，但如果执行完第一步后，计算机发生故障，则第二步永远不会执行，就会形成数据库的不一致性

◆ 问题的解决：必须有事务的概念和原子性

### ● 完整性检查和保证

◆ 系统自动在删除主表元组时删除子表对应的元组

◆ 系统检查子表中是否有相应的元组，如果存在，禁止删除动作





# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结

## 4.5 视图

### □视图的特点

- 虚表，是从一个或几个基本表（或视图）导出的表
- 只存放视图的定义，不会出现数据冗余
- 基表中的数据发生变化，从视图中查询出的数据也改变

### □基于视图的操作

- 视图实际上提供了一种观察数据的逻辑窗口。对视图的操作意味着对基表进行相对应的操作。但对视图的更新（插入数据、删除、修改）有一些限制
- 视图的定义是递归的，可以定义基于该视图的新视图

# 4.5.1 视图的定义

## 1. 建立视图

**CREATE VIEW <视图名> [(<列名> [, <列名>]...)]**  
**AS <子查询>**  
**[WITH CHECK OPTION];**

- SELECT 语句表示子查询，但通常不允许包括 ORDER BY子句和DISTINCT短语
- WITH CHECK OPTION: 通过视图进行增删改操作时，不得破坏视图定义中的谓词条件（即子查询中的条件表达式）
- DBMS执行CREATE VIEW语句时只是把视图的定义存入数据字典，并不执行其中的SELECT语句。在对视图查询时，按视图的定义从基本表中将数据查出

# 1. 建立视图

## □组成视图的属性列名全部省略或全部指定

- 省略：

由子查询中SELECT目标列中的诸字段组成

- 在下列情况下，必须命名CREATE VIEW中的列：

- 1) 当列是从算术表达式、函数或常量派生的
- 2) 两个或更多的列可能会具有相同的名称（通常是因为连接）
- 3) 需要在视图中为某个列启用新的更合适的名字

# 1. 建立视图

## □ WITH CHECK OPTION选项

- 通过视图插入、删除或修改元组时检查元组是否满足视图定义中的条件，如果不满足则拒绝执行
- 如果视图定义中含有条件，建议选择WITH CHECK OPTION选项，以约束更新的数据

□[例4-41] 建立年龄小于23岁的学生视图，并要求数据更新时进行检查

```
CREATE VIEW Sage_23  
AS SELECT * FROM Student  
WHERE Sage < 23  
WITH CHECK OPTION;
```

- 当通过视图更新学生元组时，系统将检查所更新的学生年龄是否小于23岁，不满足条件时系统将拒绝执行更新操作

# 带表达式的视图

## □[例4-42] 按系建立学生平均年龄的视图

```
CREATE VIEW D-Sage(Sdept, Avgage)
AS SELECT Sdept, AVG(Sage)
FROM Student
GROUP BY Sdept;
```

## □带表达式的视图必须明确定义组成视图的各个属性列名

- 因在SELECT目标表中有聚集函数AVG，视图定义中必须含有列名选项
- 视图的列名与SELECT后的列名相对应，即使有与基本表相同的列名也不能省略

# 基于多个基表的视图



□[例4-43] 建立计算机系选修了2号课的学生姓名和成绩的视图

```
CREATE VIEW CS_SC(Sno, Sname, Grade)
```

```
AS SELECT Student.Sno, Sname, Grade
```

```
FROM Student, SC
```

```
WHERE Sdept='CS' AND
```

```
Student.Sno=SC.Sno AND SC.Cno='2';
```

□在SELECT目标表中有带表名的列名 Student.Sno，视图定义中必须要重新定义列名

□[例4-44] 建立计算机系选修了2号课且成绩在90分以上的学生视图

```
CREATE VIEW CS_90  
AS SELECT Sno, Sname, Grade  
FROM CS_SC  
WHERE Grade>=90;
```



# 建立视图 (续)

## □不易扩充的视图:

- 以SELECT \*方式创建的视图可扩充性差, 应尽可能避免

## □[例] 将Student表中所有女生记录定义为一个视图

**CREATE VIEW**

**F\_Student1(stdnum, name, sex, age, dept)**

**AS SELECT \* FROM Student**

**WHERE Ssex='女';**

**SELECT Sno, Sname, Ssex, Sage, Sdept**

## □缺点

- 修改基表Student的结构后, Student表与F\_Student1视图的映象关系被破坏, 导致该视图不能正确工作

## 2. 删除视图

### □ DROP VIEW <视图名>;

- 该语句从数据字典中删除指定的视图定义
- 删除基表时，由该基表导出的所有视图定义都必须显式删除

### □ [例4-45] 删除学生视图CS\_90

**DROP VIEW CS\_90;**

## 4.5.2 视图上的操作

- 从用户角度：与基本表一样，通过视图可以对数据库执行查询和更新操作
- DBMS实现视图查询的方法
  - ① 实体化视图 (View Materialization)
    - ◆ 有效性检查：检查所查询的视图是否存在
    - ◆ 执行视图定义，将视图临时实体化，生成临时表
    - ◆ 查询视图转换为查询临时表
    - ◆ 查询完毕删除被实体化的视图（临时表）

## 4.5.2 视图上的操作

### □ DBMS实现视图查询的方法

#### ② 视图消解法 (View Resolution)

- ◆ 进行有效性检查，检查查询的表、视图等是否存在。如果存在，则从数据字典中取出视图的定义
- ◆ 把视图定义中的子查询与用户的查询结合起来，转换成等价的对基本表的查询
- ◆ 执行修正后的查询

# 1. 查询

## □[例4-46] 查询计算机系年龄小于23岁的学生

```
SELECT * FROM Sage_23  
WHERE Sdept='CS';
```

- 视图消解法：系统首先从数据字典中取出视图定义，把查询语句与视图定义中的子查询合并在一起，然后在相关基本表上执行查询。转换后的查询语句为：

```
SELECT * FROM Student  
WHERE Sdept='CS' AND Sage<23;
```

## □视图消解法的局限

- 有些情况下，视图消解法不能生成正确查询。采用视图消解法的DBMS会限制这类查询
- 简单视图的视图消解是总能进行的。但含有聚集函数的视图消解可能会发生错误，只能采取对基表直接操作的方式

# 1. 查询

- [例4-47] 查询所在系，要求学生的平均年龄小于21岁，该查询在学生平均年龄视图D-Sage上执行

```
CREATE VIEW D-Sage(Sdept, Avgage)
AS SELECT Sdept, AVG(Sage)
FROM Student GROUP BY Sdept;

SELECT Sdept
FROM D-Sage
WHERE Avgage<21;
```

- 其转换后的等价语句为：

```
SELECT Sdept
FROM Student
GROUP BY Sdept
HAVING AVG(Sage)<21;
```

## 2. 更新

- 对视图的数据插入、删除、修改最终转换为对基本表的操作来进行
- 用户角度：更新视图与更新基本表相同
- 指定WITH CHECK OPTION子句后，DBMS在更新视图时会进行检查，防止用户通过视图对**不属于视图范围内**的基本表数据进行更新
- 有些视图是不可更新的
  - 因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新

## 2. 更新

□[例4-48] 通过视图Sage\_23插入学生刘敏的信息  
(‘20041’, ‘刘敏’, 21, ‘女’, ‘MA’)

```
CREATE VIEW Sage_23
AS SELECT * FROM Student
WHERE Sage < 23
WITH CHECK OPTION;
INSERT INTO Sage_23
VALUES (‘20041’, ‘刘敏’, 21, ‘女’, ‘MA’);
```

- 以上插入将转换成如下语句执行:

```
INSERT INTO Student
VALUES (‘20041’, ‘刘敏’, 21, ‘女’, ‘MA’);
```



## 2. 更新

□[例4-50] 通过视图Sage\_23删除学生王茵的记录

**DELETE FROM Sage\_23 WHERE Sname='王茵';**

- 该删除语句将转换为对基本表的操作:

**DELETE FROM Student**

**WHERE Sname='王茵' AND Sage<23;**

## 2. 更新



□[例4-52] 通过视图Sage\_23修改学生王茵的年龄为22岁

```
UPDATE Sage_23  
SET Sage=22  
WHERE Sname='王茵';
```

- 该修改转换为对学生表的修改:

```
UPDATE Student  
SET Sage=22  
WHERE Sname='王茵';
```

- 因修改后学生年龄小于23岁，该操作可直接对表Student修改

# 更新视图的限制

- 一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新（对两类方法均如此）
- **[例4-49]** 通过视图D-Sage插入计算机系学生的平均年龄（‘CS’, 21）

**INSERT INTO D-Sage**

**VALUES (‘CS’, 21);**

- 无法将其转换成对基本表的更新，视图D-Sage为不可更新视图

# 更新视图的限制

□[例4-51] 通过视图CS\_SC删除学生刘明亮的信息

```
CREATE VIEW CS_SC(Sno, Sname, Grade)
```

```
AS SELECT Student.Sno, Sname, Grade
```

```
FROM Student, SC
```

```
WHERE Sdept='CS' AND
```

```
Student.Sno=SC.Sno AND SC.Cno='2';
```

```
DELETE FROM CS_SC
```

```
WHERE Sname='刘明亮';
```

不明确是删除学生刘明亮的信息还是该学生的选课信息。删除操作涉及两个表，是不能执行的

# 实际系统对视图更新的限制

- 视图更新是一个比较复杂的问题，在实际商品化系统中对视图的更新都有限制
- 有的视图是不可更新的，但也有一些视图是可更新的而实际系统没有实现
  - **不可更新的视图**：理论上证明不可更新的视图
  - **不允许更新的视图**：实际的数据库系统不支持更新，但其本身从理论上可以更新的视图
- 仅在一个表上取其行列值且其列中包含了候选键，这样所形成的视图都是可更新的，这类视图称为 **“行列子集视图”**
- 而对其它视图的更新会受到限制



# 实际系统对视图更新的限制



## □DB2对视图更新的限制：

- (1) 若视图是由两个以上基本表导出的，则此视图不允许更新
- (2) 若视图的字段来自字段表达式或常数，则不允许对此视图执行INSERT和UPDATE操作，但允许执行DELETE操作
- (3) 若视图的字段来自集函数或含有GROUP BY子句、含有DISTINCT短语，则此视图不允许更新
- (4) 若视图定义中有嵌套查询，并且内层查询的FROM子句中涉及的表也是导出该视图的基本表，则此视图不允许更新
- (5) 一个不允许更新的视图上定义的视图也不允许更新

## 4.5.3 视图的优点

### □(1) 视图提供了数据的逻辑独立性

- 例：数据库逻辑结构发生改变

- ◆ 关系 Student(Sno, Sname, Ssex, Sage, Sdept) 拆分：

SX(Sno, Sname, Sage) 和 SY(Sno, Ssex, Sdept)

- ◆ 通过建立一个视图Student：

```
CREATE VIEW Student(Sno, Sname, Ssex, Sage,  
Sdept) AS SELECT SX.Sno, SX.Sname, SY.Ssex,  
SX.Sage, SY.Sdept  
FROM SX, SY  
WHERE SX.Sno=SY.Sno;
```

- ◆ 使用户的外模式保持不变，从而对原Student表的查询程序不必修改

## 4.5.3 视图的优点

### □(1) 视图提供了数据的逻辑独立性

- 视图在一定程度上（只能在一定程度上）保证了数据的逻辑独立性
  - ◆ 由于视图更新是有条件的，因此应用程序中修改数据的语句可能仍会因基本表结构的改变而改变

### □(2) 简化了用户视图

- 定义视图能够简化用户的操作，适当地利用视图可以更清晰地表达查询
  - ◆ 基于多张表连接形成的视图
  - ◆ 基于复杂嵌套查询的视图
  - ◆ 含导出属性的视图



## 4.5.3 视图的优点

- (3) 视图使用户以不同角度看待相同的数据
  - 视图机制能使不同用户以不同方式看待同一数据，适应数据库共享的需要
- (4) 视图提供了安全保护功能
  - 对不同用户定义不同视图，使每个用户只能看到他有权看到的数据
  - 通过WITH CHECK OPTION对关键数据定义操作限制



# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结

## 4.6 SQL的数据控制

### □ SQL的数据控制功能包括

- 数据的安全性 —— 第6章
- 数据的完整性 —— 第7章
- 数据恢复 —— 第8章
- 并发控制 —— 第9章

### □ 本章介绍安全性中对数据的存取权限控制语句

- SQL中，权限通常是指使用SQL语句存取数据的权力，SQL提供了一套灵活的授权机制，哪些用户拥有对哪些数据的操作权限可以通过授权获得
  - ◆ 授权语句GRANT
  - ◆ 回收语句REVOKE

## 4.6.1 授权

□ 授权是指有授予权的用户将自己所拥有的权限授予其他用户

□ 授权语句格式为：

**GRANT <权限1> [, <权限2>] ...**

**ON TABLE <表名或视图名>**

**TO {<用户名1> [, <用户名2>] ... | PUBLIC}**

**[WITH GRANT OPTION];**

- 当有WITH GRANT OPTION短语时，被授权的用户还可以把获得的权限再授予其它用户
- 一个GRANT语句可以把相应权限同时授予多个用户，用户名表如用短语PUBLIC代替，则表示把权限授予所有数据库的用户

## 4.6.1 授权

□[例4-55] 授予用户User2在表Student上的查询权和删除权，同时使User2拥有将所得权限授予其他用户的权力

**GRANT SELECT, DELETE**

**ON TABLE Student**

**TO User2**

**WITH GRANT OPTION;**

- 执行以上语句后，用户User2获得了对表Student的查询权和删除权，同时也获得在表Student上的授予权

## 4.6.2 权限回收

- 具有授予权的用户可以通过回收语句将所授予的权限回收
- 回收语句格式为：

```
REVOKE <权限1> [, <权限2>] ...  
ON TABLE <表名或视图名>  
FROM {<用户名1> [, <用户名2>] ... | PUBLIC}  
[RESTRICT | CASCADE];
```

- CASCADE选项表示回收权限时要引起级联操作，即拥有授予权 (WITH GRANT OPTION) 的用户如果把拥有的权限授予了其他用户，则要把转授出去的权限一起回收
- RESTRICT选项表示，只有用户没有将拥有的权限转授给其他用户时才能回收该用户的权限，否则系统将拒绝执行



## 4.6.2 权限回收

□[例4-58] 回收用户User2对学生表Student的查询权和删除权

```
REVOKE SELECT, DELETE  
ON TABLE Student  
FROM User2  
CASCADE;
```



# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结



## 4.7 嵌入式SQL

- 4.7.1 嵌入式SQL与主语言的接口
- 4.7.2 不用游标的嵌入式SQL
- 4.7.3 用游标的嵌入式SQL
- 4.7.4 嵌入式SQL应用实例
- 4.7.5 动态SQL

## 4.7 嵌入式SQL

- SQL语言提供了两种使用方式：交互式、嵌入式
- 为什么要引入嵌入式SQL：
  - SQL是面向集合的，是非过程性的。而许多事务处理是过程性的，与上下文相关的，单纯使用SQL语句难以实现各种应用的全部功能，需要应用高级语言
- 为解决这一问题，SQL语言提供了嵌入式使用方式
  - 将SQL语言嵌入到高级语言中，利用高级语言的结构性来弥补SQL语言实现复杂应用方面的不足，称为嵌入式SQL (Embedded SQL)，而嵌入SQL的高级语言称为主语言或宿主语言



## 4.7.1 嵌入式SQL与主语言的接口



- 将SQL语句嵌入到宿主语言中必须解决的问题
  - (1) 编译嵌入主语言的SQL语句成为可执行代码
  - (2) 数据库和主语言程序间的通信
  - (3) 数据库和主语言程序间的数据交换
  - (4) 需要协调面向集合和面向记录两种不同的处理方式

# (1) 嵌入式SQL的编译

## □采用预编译方法

- 在编译之前，先对SQL语句进行预处理
- 通过执行预处理程序把SQL语句变为主语言能够识别的形式
- 由主语言编译器统一对预处理后的源程序进行编译

## □预处理要求：

- 所有SQL语句必须加前缀**EXEC SQL**，以区分SQL语句与主语言语句
- 有的主语言还在SQL语句结尾加上结束标志
- 在C语言嵌入式SQL的一般格式为：

**EXEC SQL <SQL语句>;**



## (2) 数据库和主语言程序间的通信



- SQL语句执行是否成功，需要将执行结果反馈给应用程序
  - 在SQL中设有一通信区SQLCA (SQL Communication Area)
    - ◆ SQLCA是一个数据结构，其中包含描述DBMS当前工作状态的若干信息
  - 应用程序和用户通过SQLCA了解数据库的执行情况
    - ◆ SQLCA中有一状态指示单元SQLCODE，用于存放SQL语句的执行结果，通过判断SQLCODE的值可以知道SQL语句的执行情况（0表示成功）



## (2) 数据库和主语言程序间的通信



### □ SQLCA的内容

- 与所执行的SQL语句有关
- 与该SQL语句的执行情况有关

例：在执行删除语句DELETE后，不同的执行情况，SQLCA中有不同的信息：

- ◆ 违反数据保护规则，操作拒绝
- ◆ 没有满足条件的行，一行也没有删除
- ◆ 成功删除，并有删除的行数
- ◆ 无条件删除警告信息
- ◆ 由于各种原因，执行出错

### (3) 数据库和主语言程序间的数据交换



□ 嵌入式SQL语句有输入变量和输出变量，用来与主语言进行数据交换

- 这些变量是主语言定义的程序变量，简称**主变量** (host variable) 或宿主变量

□ 主变量的用途

- 输入主变量
  - ◆ 指定向数据库中插入的数据；将数据库中的数据修改为指定值；指定执行的操作
  - ◆ 指定WHERE子句或HAVING子句中的条件
- 输出主变量
  - ◆ 获取SQL语句的结果数据和SQL语句的执行状态

## (4) 面向集合和面向记录的协调

### □ 集合性操作语言与过程性操作语言的不匹配

- SQL语言是面向集合的，一条SQL语句原则上可以产生或处理多条记录
- 主语言是面向记录的，一组主变量一次只能存放一条记录
- 仅使用主变量并不能完全满足SQL语句向应用程序输出数据的要求

### □ 嵌入式SQL引入了游标 (Cursor) 的概念，用来协调这两种不同的处理方式

- 游标是系统为用户开设的一个数据缓冲区，存放SQL语句的执行结果
- 每个游标区都有一个名字，游标是可移动的



## 4.7.2 不用游标的嵌入式SQL



- 如果嵌入式SQL语句的执行结果为单记录，或一次处理多个记录而不是对记录逐个处理，可以不用游标
- 不用游标的SQL语句的种类
  - 说明性语句
  - 数据定义语句
  - 查询结果为单记录的SELECT语句
  - INSERT语句
  - 非CURRENT形式的UPDATE语句
  - 非CURRENT形式的DELETE语句
  - 授权和回收语句

# 1. 说明性语句

□[例4-59] 定义字符型主变量Sno和Sname

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
CHAR Sno(6);
```

```
CHAR Sname(8);
```

```
... ..
```

```
EXEC SQL END DECLARE SECTION;
```

□所有主变量在使用前需用DECLARE语句说明

## 2. 查询结果为单记录的SELECT语句



- 查询结果为单记录的SELECT语句可以不用游标，但要有INTO子句将查询结果放入主变量
- **[例4-60]** 根据学号查询学生姓名和选修2号课的成绩

**EXEC SQL SELECT Sname, Cno, Grade**

**INTO :Hsname, :Hcno, :Hgrade**

**FROM Student, SC**

**WHERE Student.Sno = :Hsno AND Cno='2'**

**AND Student.sno=SC.sno;**

- Hsno为输入主变量，在执行该语句前，应先用主语言命令给Hsno赋值

# 3. INSERT语句

□[例4-61] 在学生选课表中插入选课成绩

EXEC SQL INSERT INTO SC

VALUES (:Sno, :Cno, :Grade);

□在执行该语句前，应先给主变量赋值，然后执行该语句，将选课成绩插入SC表

## 4. 非CURRENT形式的更新语句

### □在嵌入式SQL中的更新语句

- 可以一次修改多个元组
- 也可以先查询出结果集，然后根据应用需要逐个修改，需要与游标配合使用

### □非CURRENT形式的更新语句：不用游标的更新语句

### □[例4-62] 给出学生的学号和要修改的课程号，修改学生选修记录

```
EXEC SQL UPDATE SC SET Cno=:Hnewcno  
WHERE Sno=:Hsno AND Cno=:Holdcno;
```

- 主变量Holdcno中为修改前的课程号，Hnewcno中为修改后的课程号

## 4.7.3 用游标的嵌入式SQL

□ 高级语言一次只能处理一条记录，而当处理的结果集是多条时，就必须使用游标机制，从而把对集合的操作转化为对单个记录的处理

□ 使用游标的步骤

- 1. 定义游标
- 2. 打开游标
- 3. 移动游标指针并取当前记录
- 4. 关闭游标

| <i>Statement</i> |
|------------------|
| DECLARE CURSOR   |
| OPEN             |
| FETCH            |
| CLOSE            |
| DEALLOCATE       |

# 1. 定义游标

□使用DECLARE语句

□语句格式

```
EXEC SQL DECLARE <游标名> CURSOR  
FOR <SELECT 语句>  
[FOR UPDATE OF <列名>];
```

□功能

- 是一条说明性语句，这时DBMS并不执行指定操作

## 2. 打开游标

□使用OPEN语句

□语句格式

**EXEC SQL OPEN** <游标名>;

□功能

- 打开游标实际上是执行相应的查询语句，把所有满足查询条件的记录从指定表取到缓冲区中
- 这时游标处于活动状态，指针指向查询结果集中**第一条记录之前**的位置





# 3. 移动游标指针并取当前记录



□使用FETCH语句，语句格式

**EXEC SQL** **FETCH** <游标名>

**INTO** <主变量> [, <主变量>]...;

□功能

- 指定方向推动游标指针，然后将缓冲区中的当前记录取出来送至主变量供主语言进一步处理
- NEXT|PRIOR|FIRST|LAST：指定游标移动方式

□说明

- 主变量与SELECT语句中的目标列表达式一一对应
- FETCH语句通常用在一个循环结构中，通过循环执行FETCH语句逐条取出结果集中的行进行处理
- 为进一步方便用户处理数据，现在一些关系数据库管理系统对FETCH语句做了扩充，允许用户向任意方向以任意步长移动游标指针

## 4. 关闭游标

□使用CLOSE语句

□语句格式

**EXEC SQL CLOSE** <游标名>;

□功能

- 关闭游标，释放结果集占用的缓冲区及其他资源

□说明

- 游标被关闭后，就不再和原来的查询结果集相联系
- 被关闭的游标可以再次被打开，与新的查询结果相联系

# 游标的工作原理

```
1 EXEC SQL BEGIN ..... //定义主变量
2 EXEC SQL DECLARE tnames_cursor CURSOR
  FOR SELECT name,xh FROM xjb
    WHERE xb='男'
    ORDER BY xh;
3 EXEC SQL OPEN tnames_cursor;
4 EXEC SQL FETCH tnames_cursor INTO :PNAME,:PXH;
  WHILE (PXH!=NULL)
  {
    4 printf("%s,%s\n",PNAME,PXH);
    EXEC SQL FETCH tnames_cursor INTO :PNAME,:PXH;
  }
5 EXEC SQL CLOSE tnames_cursor;
EXEC SQL DEALLOCATE tnames_cursor;
```

| 2 | :PNAME | :PXH |
|---|--------|------|
| 3 | 张三     | 003  |
|   | 李四     | 004  |
|   | 王五     | 005  |
|   | 孙六     | 006  |
|   | 陈七     | 007  |
|   | 曹八     | 008  |
|   | .      |      |
|   | .      |      |
|   | .      |      |

## 4 Results

|     |     |
|-----|-----|
| 张三  | 003 |
| 李四  | 004 |
| 王五  | 005 |
| 孙六  | 006 |
| 陈七  | 007 |
| 曹八  | 008 |
| ... |     |



## □用途

- 非CURRENT形式的UPDATE语句和DELETE语句
  - ◆ 面向集合的操作
  - ◆ 一次修改或删除所有满足条件的记录
- 如果只想修改或删除其中某个记录
  - ◆ 用带游标的SELECT语句查出所有满足条件的记录
  - ◆ 从中进一步找出要修改或删除的记录
  - ◆ 用CURRENT形式的UPDATE语句和DELETE语句修改或删除之

- (1) DECLARE说明游标
- (2) OPEN打开游标，把所有满足查询条件的记录从指定表取至缓冲区
- (3) FETCH推进游标指针，并把当前记录从缓冲区中取出来送至主变量
- (4) 检查该记录是否是要修改或删除的记录，是则处理
- (5) 重复第(3)和(4)步，用逐条取出结果集中的行进行判断和处理
- (6) CLOSE关闭游标，释放结果集占用的缓冲区和其他资源
  - 当游标定义中的SELECT语句带有UNION或ORDER BY子句时，或者该SELECT语句相当于定义了一个不可更新的视图时，不能使用CURRENT形式的UPDATE语句和DELETE语句

## 4.7 嵌入式SQL

- 4.7.1 嵌入式SQL与主语言的接口
- 4.7.2 不用游标的嵌入式SQL
- 4.7.3 用游标的嵌入式SQL
- 4.7.4 嵌入式SQL应用实例
- 4.7.5 动态SQL

## □静态SQL的特点

- 用户可以在程序运行过程中根据实际需要输入WHERE子句或HAVING子句中某些变量的值
- 语句中主变量的个数与数据类型在预编译时都是确定的，只有是主变量的值是程序运行过程中动态输入的

## □静态SQL的不足

- 静态SQL语句提供的编程灵活性在许多情况下仍显得不足，不能编写更为通用的程序

## □动态SQL语句

- 程序运行期间临时组装SQL语句，而不是像静态SQL语句一样，在程序运行前语句已经确定，运行期间只需要确定主变量的值

## □动态SQL语句主要有三种形式：

- 语句可变：程序运行期间临时输入完整的SQL语句
- 条件可变：WHERE条件可以根据程序运行状态改变形式
- 数据库对象、查询条件均可变：SELECT的列名、FROM的数据表名、WHERE的条件均可以由用户临时构造



## □ 动态SQL的执行方式

- 即席输入后直接执行

**EXEC SQL EXECUTE IMMEDIATE <主变量1>;**

- 经过预处理后多次执行

预处理语句格式:

**EXEC SQL PREPARE <语句名> FROM <主变量1>;**

执行语句格式:

**EXEC SQL EXECUTE <语句名> [USING <主变量2>];**

- ◆ 主变量1中存放可执行的SQL语句
- ◆ 主变量2给出与语句名关联的SQL语句的参数

# 动态嵌入式SQL



## □ 直接执行

```
scanf ("%s", dstring);
```

```
dstring="DELETE FROM SC WHERE Sno='S01'";
```

```
EXEC SQL EXECUTE IMMEDIATE :dstring;
```

## □ 预处理执行

```
strcpy(dstring, "UPDATE SC SET Grade=:g WHERE  
Cno='C005'");
```

```
EXEC SQL PREPARE S1 FROM :dstring;
```

```
EXEC SQL EXECUTE S1 USING :HGrade;
```

## 4.7 嵌入式SQL

- 在嵌入式SQL中，SQL语句与主语言语句分工非常明确
  - SQL语句
    - ◆ 直接与数据库打交道，取出数据库中的数据
    - ◆ 面向集合，一条SQL语句可以产生或处理多条记录
  - 主语言语句
    - ◆ 控制程序流程
    - ◆ 对取出的数据做进一步加工处理
    - ◆ 主语言是面向记录的，一组主变量一次只能存放一条记录
  - 仅使用主变量并不能完全满足SQL语句向应用程序输出数据的要求
  - 嵌入式SQL引入了游标，协调两种不同的处理方式

## 4.8 小结

### □ SQL的特点

- 1. 综合统一
- 2. 高度非过程化
- 3. 面向集合的操作方式
- 4. 同一种语法结构提供两种使用方式

### □ 交互式SQL

- 数据定义
- 数据查询
- 数据操纵
- 数据控制

| SQL功能 | 动词                     |
|-------|------------------------|
| 数据查询  | SELECT                 |
| 数据定义  | CREATE, DROP, ALTER    |
| 数据操纵  | INSERT, UPDATE, DELETE |
| 数据控制  | GRANT, REVOKE          |

## 4.8 小结

### □ 数据查询

- 对表的选取 (FROM)
- 对列的选取
  - ◆ 目标表达式、更改列标题
- 对行的选取 (WHERE)
  - ◆ 消除重复行、WHERE子句条件表达式
- 排序 (ORDER BY)
- 聚集函数
- 分组 (GROUP BY, GROUP BY...HAVING)

## 4.8 小结

### □ 数据查询 (续)

- WHERE子句的条件表达式
  - ◆ 比较操作符
  - ◆ 确定范围
  - ◆ 集合查找
  - ◆ 字符匹配: 各种匹配符
  - ◆ 空值运算
  - ◆ 多重条件组合

## 4.8 小结

### □ 数据查询 (续)

- 多表连接查询

- ◆ 等值连接、自然连接

- ◆ 外连接

- ◆ 复合条件连接

- ◆ 子查询

- 不相关子查询、相关子查询

- ◆ 集合查询

## 4.8 小结

### □ 数据操纵

- 插入数据
- 修改数据
  - ◆ 更新单表数据
  - ◆ 带子查询的数据修改
- 删除数据
  - ◆ 删除单表数据
  - ◆ 带子查询的数据删除



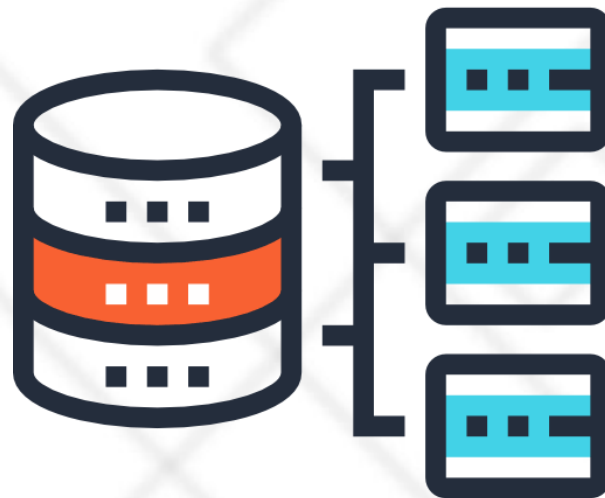
## 4.8 小结

### □视图

- 定义方法
- 使用视图
  - ◆ WITH CHECK OPTION
- 视图消解



北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY



# 作业

《数据库系统原理》第4章

习题4, 6

## □实验2：标准SQL语言和简单查询

### □实验目的

- 掌握SQL程序设计基本规范、熟练运用SQL语言实现数据基本查询

### □实验内容

- 详见乐学课程

### □提交方式

- 打包提交：实验报告、源文件（.sql文件）
- 压缩文件命名方式：学号-姓名-实验X